

Development Automation **& Linux**

Lab File

Submitted By:

Ayush Singh Kushwah

SAP ID: 500095575

Enrolment No: R2142210216

Batch: 03

Semester IV

BTech CSE (DevOps)

Submitted To:

Mr. Suvra Choudhary



SCHOOL OF COMPUTER SCIENCE
UNIVERSITY OF PETROLEUM & ENERGY STUDIES
Dehradun-248007
2022-23

INDEX

S.No	Experiment	Date	Remark
•			
1.			
2.			
3.			
4.			
5.			
6.			
7.			
8.			
9.			
10.			
11.			
12.			

EXPERIMENT-1

Introduction to Linux Commands

1. Write brief description of the following Shell commands. Also, attach screenshot after running these commands on the terminal.

- man: an interface to the system manuals.

```
MAN(1)                                Manual pager utils                                MAN(1)

NAME
    man - an interface to the system reference manuals

SYNOPSIS
    man [man options] [[section] page ...] ...
    man -k [apropos options] regexp ...
    man -K [man options] [section] term ...
    man -f [whatis options] page ...
    man -l [man options] file ...
    man -w|-W [man options] page ...

DESCRIPTION
    man is the system's manual pager. Each page argument given to man is normally the name of a program, utility or function. The manual page associated with each of these arguments is then found and displayed. A section, if provided, will direct man to look only in that section of the manual. The default action is to search in all of the available sections following a pre-defined order (see DEFAULTS), and to show only the first page found, even if page exists in several sections.

    The table below shows the section numbers of the manual followed by the types of pages they contain.

    1 Executable programs or shell commands
    2 System calls (functions provided by the kernel)
    3 Library calls (functions within program libraries)
    4 Special files (usually found in /dev)
    5 File formats and conventions, e.g. /etc/passwd
    6 Games
    7 Miscellaneous (including macro packages and conventions), e.g. man(7), groff(7), man-pages(7)

Manual page man(1) line 1 (press h for help or q to quit)
```

- whoami: prints the user name associated with the current effective user ID.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ whoami
ayroid
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- pwd: prints name of the current working directory.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ pwd
/mnt/c/college/upes/sem 4/linux/lab
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- mkdir: used to create new directories

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
Experiments  Lab File  Notes  a.txt  expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ mkdir newdir
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
Experiments  Lab File  Notes  a.txt  expl.txt  newdir
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- cd: used to change directories

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ pwd
/mnt/c/college/upes/sem 4/linux/lab
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ cd notes
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/notes$ pwd
/mnt/c/college/upes/sem 4/linux/lab/notes
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/notes$
```

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/notes$ pwd
/mnt/c/college/upes/sem 4/linux/lab/notes
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/notes$ cd ..
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ pwd
/mnt/c/college/upes/sem 4/linux/lab
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- ls: used to list the contents of the directory.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
Experiments  Lab File  Notes  a.txt  expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- ls -l: used to list the contents of the directory in a long listing format.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls -l
total 0
drwxrwxrwx 1 ayroid ayroid 4096 Jan 26 18:35 Experiments
drwxrwxrwx 1 ayroid ayroid 4096 Jan 26 18:43 Lab File
drwxrwxrwx 1 ayroid ayroid 4096 Jan 26 18:25 Notes
-rwxrwxrwx 1 ayroid ayroid 81 Jan 17 23:54 a.txt
-rwxrwxrwx 1 ayroid ayroid 151 Jan 20 10:27 expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- ls -a: shows all contents of the directory + entries starting with .

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux$ ls -a
.  ..  lab
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux$
```

- ls -c: sorts and shows the content of the directory according to name/time when combined with l/lt respectively.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls -c
Lab File  Experiments  Notes  expl.txt  a.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls -cl
total 0
drwxrwxrwx 1 ayroid ayroid 4096 Jan 26 18:35 Experiments
drwxrwxrwx 1 ayroid ayroid 4096 Jan 26 19:29 Lab File
drwxrwxrwx 1 ayroid ayroid 4096 Jan 26 18:25 Notes
-rwxrwxrwx 1 ayroid ayroid 81 Jan 17 23:54 a.txt
-rwxrwxrwx 1 ayroid ayroid 151 Jan 20 10:27 expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls -clt
total 0
drwxrwxrwx 1 ayroid ayroid 4096 Jan 26 19:29 Lab File
drwxrwxrwx 1 ayroid ayroid 4096 Jan 26 18:35 Experiments
drwxrwxrwx 1 ayroid ayroid 4096 Jan 26 18:25 Notes
-rwxrwxrwx 1 ayroid ayroid 151 Jan 20 10:27 expl.txt
-rwxrwxrwx 1 ayroid ayroid 81 Jan 17 23:54 a.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- ls -m: shows the content of the directory separated by commas

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls -m
Experiments, Lab File, Notes, a.txt, expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- ls -p: shows the content of the directory with / indicating directories

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls -p
Experiments/ Lab File/ Notes/ a.txt expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- ls -r: shows the content of the directory by sorting them in reverse

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls -r
expl.txt a.txt Notes Lab File Experiments
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

ls -f: shows the content of the directory without sorting or colouring them

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls -f
.  ..  a.txt  expl.txt  Experiments  Lab File  Notes
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- clear: clears the terminal screen

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
experiments  lab file  notes  a.txt  expl.txt  newdir
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ rmdir newdir
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
experiments  lab file  notes  a.txt  expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ clear
```

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- rmdir: removes empty directories

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
experiments  lab file  notes  a.txt  expl.txt  newdir
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ rmdir newdir
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
experiments  lab file  notes  a.txt  expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- touch: creates new files without opening the text editor

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
experiments  lab file  notes  a.txt  expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ touch b.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
experiments  lab file  notes  a.txt  b.txt  expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- cat: concatenates files and prints content on the screen

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ cat expl.txt
man
whoami
ls (a, l, m, c, p, r)
ls -l
ls -a
```

- less: shows file's contents one screen at a time. It doesn't load the entire file but accesses it page by page.

```
1
2
3
4
5
6
7
8
9
10
```

- more: used to view the first section of the file.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ more c.txt
1
2
3
4
5
6
7
8
9
10
```

- rm: removes files and directories

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
experiments  lab file  notes  a.txt  b.txt  expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ rm b.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
experiments  lab file  notes  a.txt  expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- sort: sorts lines of the text files

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ sort a.txt
1
10
11
12
13
14
15
16
17
18
19
2
```

- cp: copies files and directories

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls ..
lab
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
Experiments Lab File Notes a.txt expl.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ cp a.txt /mnt/c/college/upes/'sem 4'/linux
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls ..
lab a.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- mv: moves and renames files

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
Experiments Lab File Notes a.txt expl.txt scripts
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls ..
lab
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ mv a.txt /mnt/c/college/upes/'sem 4'/linux
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls ..
lab a.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
Experiments Lab File Notes a.txt expl.txt scripts
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ mv a.txt c.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ ls
Experiments Lab File Notes c.txt expl.txt scripts
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- wc: prints newline, word, and byte counts for each file

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ wc c.txt
30 30 81 c.txt
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- head: prints first n lines of the file. The default value of n is 10.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ head c.txt
1
2
3
4
5
6
7
8
9
10
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- tail: prints the last n lines of the file. The default value of n is 10.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ tail c.txt
21
22
23
24
25
26
27
28
29
30
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- history: GNU history library. Keeps record of the previously used commands.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ history
1  ls
2  cd ..
3  ls
4  cd home
5  ls
6  cd ayroid
7  ls
8  cd ..
9  ls
10 cd ..
```

2. Write in details about the grep command.

grep searches for PATTERNS in each FILE. PATTERNS is one or more patterns separated by newline characters, and grep prints each line that matches a pattern. Typically, PATTERNS should be quoted when grep is used in a shell command.

- i) Find all the patterns that start with “ep” in a given file.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ grep ep new.txt
ep
episode
hepi
episodes
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- ii) Print those lines of a given text file which do not contain the word “hi”.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ grep -v hi new.txt
ep
episode
hahah
hepi
episode
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- iii) Explain the usage of options -i, -A, -B, -C in grep command with examples.

- grep -i: ignores case while looking for a match.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ grep -i "hi" new.txt
hi
Hisikes
phiness
KrHies
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$
```

- grep -A n: prints the matched results and n number of lines after that.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ grep -A 1 "hi" new.txt
hi
ep
--
phiness
KrHies
```

- grep -B n: prints the matched results and n number of lines before that.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ grep -B 1 "hi" new.txt
hi
--
Hisikes
phiness
```

- grep -C n: prints the matched results and n number of lines before & after that.

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab$ grep -C 1 "hi" new.txt
hi
ep
--
Hisikes
phiness
KrHies
```

EXPERIMENT-2

Introduction to Bash Scripts

1. Write a script to enter marks by user in the terminal. If the entered mark is greater than 35 then output pass, otherwise output fail.

```
#!/bin/bash

echo "Enter Marks"
read MARKS
if [ $MARKS -gt 35 ];
then
    echo "pass"
else
    echo "fail"
fi
```

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques1.sh
Enter Marks
34
fail
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques1.sh
Enter Marks
35
fail
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques1.sh
Enter Marks
36
pass
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$
```

2. Write a script to enter marks by user in the terminal. If marks are greater than 80 output distinction, if marks between 60 and 80 then output first division. If marks are less than 60 and greater than or equal to 35 then output pass, otherwise output fail.

```
#!/bin/bash

echo "Enter Marks"
read MARKS
if [ $MARKS -gt 80 ];
then
    echo "distinction"
elif [ $MARKS -le 80 -a $MARKS -ge 60 ];
then
    echo "first division"
elif [ $MARKS -lt 60 -a $MARKS -ge 35 ];
then
    echo "pass"
else
    echo "fail"
fi
```



```

ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques2.sh
Enter Marks
81
distinction
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques2.sh
Enter Marks
80
first division
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques2.sh
Enter Marks
75
first division
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques2.sh
Enter Marks
60
first division
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques2.sh
Enter Marks
40
pass
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques2.sh
Enter Marks
34
fail
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$

```

3. Write a script to print numbers from 1 to 10 using while loop.

```

#!/bin/bash

num=1

while [ $num -le 10 ]; do
    echo $num
    num=$((num+1))
done

ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques3.sh
1
2
3
4
5
6
7
8
9
10

```

4. Write a script to find the average of first N natural numbers, where N is entered by the user at the terminal.

```

#!/bin/bash

echo "Enter Number"
read num
cpnum=num
avg=0
i=0
while [ $i -le $num ]; do
    avg=$((avg+i))
    i=$((i+1))
done
echo $avg
avg=$((avg/num))
echo "Average of first $num natural numbers is $avg"

# Bash doesn't support floating-point arithmetic

ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques4.sh
Enter Number
4
10
Average of first 4 natural numbers is 2
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$

```

5. Write a script to find the average of N numbers of an array (of size N). The values of the array are entered by the user at the terminal. (Use while loop)

```
#!/bin/bash

echo "Enter Size of Array"
read size
array=()
i=0
avg=0

echo "Enter Elements in Array"

while [ $i -lt $size ]; do
    read value
    array[i]=$value
    i=$((i+1))
    avg=$((avg+value))
done

avg=$((avg/size))

echo "Average = $avg"
```

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$ ./ques5.sh
Enter Size of Array
5
Enter Elements in Array
1
2
3
4
7
Average = 3
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts$
```

EXPERIMENT-3

Shell Scripting

1. Write a shell script to fetch two command line arguments (Integers) and then add them and display the result.

Code:

```
#!/bin/bash
echo "Sum= $((($1+$2))"
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$ ./ques1.sh 10 5
Sum= 15
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$
```

2. Write a shell script to implement a menu-driven calculator with basic functionalities of addition, subtraction, division, and multiplication. Your script will read two numbers from users and an operation and display the result.

Code:

```
#!/bin/bash

echo "Enter:"
echo "1: Addition"
echo "2: Subtraction"
echo "3: Multiply"
echo "4: Division"

read choice

echo "Enter Two Numbers:"
read num1
read num2

case $choice in
    1) echo "Sum= $((num1+num2))";;
    2) echo "Diff= $((num1-num2))";;
    3) echo "Product= $((num1*num2))";;
    4) echo "Division= $((num1/num2))";;
    *) echo "Wrong Choice";;
esac
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$ ./ques2.sh
Enter:
1: Addition
2: Subtraction
3: Multiply
4: Division
3
Enter Two Numbers:
8
2
Product= 16
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$
```

3. Write a shell script that prints all the ordinary files in the current working directory.

Code:

```
#!/bin/bash
a=$(ls)
for i in $a
do
    if [ -f $i ]
    then
        echo $i
    fi
done
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$ ./ques3.sh
1
a.txt
abc.txt
ques1.sh
ques2.sh
ques3.sh
ques4.sh
ques5.sh
ques6.sh
ques7.sh
ques8.sh
ques9.sh
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$
```

4. Write a shell script that takes a command line argument (the argument entered by the user is the absolute path of any file, directory, etc.,) and reports on whether it is a directory, a file, or something else.

Code:

```
#!/bin/bash
path=$1
if [ -d "$path" ]; then
    echo "Directory"
elif [ -f "$path" ]; then
    echo "File"
else
    echo "Something Else"
fi
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$ ./ques4.sh /mnt/c/college/upes/'sem 4'/linux/lab/scripts/exp3
Directory
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$ ./ques4.sh /mnt/c/college/upes/'sem 4'/linux/lab/scripts/exp3/abc.txt
File
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$
```

5. Write a shell script to compute the factorial of a number entered by the user.

Code:

```
#!/bin/bash
echo "Enter Number:"
read num
fact=1
while [ $num -gt 0 ]
do
    fact=$((fact*num))
    num=$((num-1))
done
echo "Factorial: $fact"
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$ ./ques5.sh
Enter Number:
6
Factorial: 720
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$
```

6. Write a shell script to generate the Fibonacci series for a positive integer N which is entered by the user.

Code:

```
#!/bin/bash

echo "Enter Limit:"
read num
a=0
b=1

echo -n "$a, $b"

while [ $num -gt 2 ]
do
    c=$((a+b))
    echo -n ", $c"
    a=$b
    b=$c
    num=$((num-1))
done
echo
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$ ./ques6.sh
Enter Limit:
10
0, 1, 1, 2, 3, 5, 8, 13, 21, 34
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$
```

7. Write a shell script to print the value of nCk (read as n choose k), where n and k are entered by the user in the terminal.

Code:

```
#!/bin/bash

echo "Enter value of n:"
read n
echo "Enter value of k:"
read k

facto() {
    fact=1
    for (( i=1 ; i<= $1 ; i++ ));
    do
        fact=$((fact*i))
    done
    echo "$fact"
}

echo "$(( ${facto} n / ( ${facto} $((n-k)) * ${facto} k )))" #Formula for nCr
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$ ./ques7.sh
Enter value of n:
10
Enter value of k:
2
45
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$
```

8. Write a shell script to find out the total instances of "the" in a given file.

Code:

```
#!/bin/bash

read -p "Enter absolute path of the file: " filepath
if ! [ -r "$filepath" ]; then
    echo "File not found!"
else
    echo "Count of 'the': $(grep -o "the" "$filepath" | wc -l)"
fi
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$ ./ques8.sh
Enter absolute path of the file: /mnt/c/college/upes/sem 4/linux/lab/scripts/exp3/abc.txt
Count of 'the': 6
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$
```

9. Write a shell script that displays a list of all the files in the current directory to which the user has read, write and execute permissions.

Code:

```
#!/bin/bash

for file in *
do
    if [ -r "$file" -a -w "$file" -a -x "$file" ]
    then
        echo "$file"
    fi
done
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$ ./ques9.sh
abc.txt
ques1.sh
ques2.sh
ques3.sh
ques4.sh
ques5.sh
ques6.sh
ques7.sh
ques8.sh
ques9.sh
ayroid@Ayroid:/mnt/c/college/upes/sem 4/linux/lab/scripts/exp3$
```

EXPERIMENT-4

Functions in Shell Scripting

1. Write a function to fetch two command line arguments (Integers) and then add them and display the result.

Code:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$ cat ques1.sh
#!/bin/bash

add(){
    echo $(( $1+$2 ))
}

add $1 $2
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$ ./ques1.sh 7 4
11
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$
```

2. Write a function to input a three-digit number from the user and check if the number is even or odd. If the number is even then return the sum of the digits of the number otherwise return the product of the given number.

Code:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$ cat ques2.sh
#!/bin/bash

sumofdig(){
    num=$1
    sum=0
    while [ $num -gt 0 ]
    do
        sum=$((sum+((num%10))))
        num=$((num/10))
    done
    echo "Sum of digits: $sum"
}

proofdig(){
    num=$1
    pro=1
    while [ $num -gt 0 ]
    do
        pro=$((pro*$(num%10)))
        num=$((num/10))
    done
    echo "Product of digits: $pro"
}

checkk(){
    num=$1
    if [ $((num%2)) -eq 0 ]; then
        sumofdig $num
    else
        proofdig $num
    fi
}

checkk $1
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$ ./ques2.sh 123
Product of digits: 6
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$ ./ques2.sh 124
Sum of digits: 7
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$
```

3. Write a function to compute the factorial of a given number N which is input by the user from the terminal.

Code:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$ cat ques3.sh
#!/bin/bash

fact(){
    f=1
    for (( i=1; i<=$1 ; i++ ));
    do
        f=$((f*i))
    done
    echo $f
}

fact $1
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$ ./ques3.sh 7
5040
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$
```

4. Write a function to read the number of lines of a given text file (starting from line 1) as specified by the user. For example, if there is a file demo.txt, and the user specifies that he needs the first 7 lines of the text then display the first 7 lines of demo.txt. Similarly, if the user specifies the last 10 lines, then the last 10 lines of demo.txt should be printed on the terminal.

Code:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$ cat ques4.sh
#!/bin/bash

lines(){
    if [[ $from == "S" || $from == "s" ]]; then
        echo $(head -n $2 $1)
    elif [[ $from == "E" || $from == "e" ]]; then
        echo $(tail -n $2 $1)
    else
        echo "Wrong Choice"
    fi
}

read -p "Enter File Name: " filename
read -p "Enter Number of Lines: " lines
read -p "Enter S for Start & E for End: " from
echo
lines $filename $lines $from
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$ ./ques4.sh
Enter File Name: test.txt
Enter Number of Lines: 5
Enter S for Start & E for End: S

Language is one of the most powerful tools humans possess. It allows us to communicate our thoughts, feelings, and ideas to others, and to understand and appreciate the thoughts, feelings, and ideas of others. Every language has its own unique vocabulary, grammar, and syntax, which reflect the culture and history of the people who speak it. Learning a new language opens up new opportunities and experiences, as well as improving cognitive abilities and enhancing cultural understanding. One of the most obvious benefits of learning a new language is the ability to communicate with people who speak that language. In today's globalized world, being able to speak more than one language is becoming increasingly important. If you can speak a second language, you can communicate with people from other countries and cultures, and you can expand your business, career, and social opportunities. For example, if you work for a multinational company, being able to speak the language of your overseas colleagues or clients can help you build better relationships and close more deals. If you travel to a foreign country, being able to speak the local language can make your experience much more enjoyable and enriching, as you will be able to understand and interact with the local culture more easily. In addition to the practical benefits of learning a new language, there are also cognitive benefits. Learning a new language requires a lot of mental effort, as you have to memorize new vocabulary, grammar rules, and pronunciation. This mental exercise has been shown to have a positive effect on cognitive abilities such as memory, attention, and problem-solving. In fact, studies have shown that people who are bilingual or multilingual are better at tasks that require mental flexibility and executive control, such as switching between tasks, inhibiting irrelevant information, and focusing attention. Learning a new language can also delay the onset of dementia and other age-related cognitive decline, as the constant mental exercise helps keep the brain active and healthy.
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp4$
```


EXPERIMENT-5

Automation Scripts to save Time & Efforts

1. Set the crontab to create text file every minute in a given directory. Note that the name of the files should be different. However, you may choose any name file name.

Code:

```
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ cat ques1.sh
#!/bin/bash

num=$RANDOM
(touch /mnt/c/college/upes/sem4/linux/lab/scripts/exp5/$num.txt)
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ █

#Creatig a text file every minute
* * * * * /mnt/c/college/upes/sem4/linux/lab/scripts/exp5/ques1.sh
```

Output (After 3 mins):

```
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ ls
13250.txt 4441.txt 9730.txt ques1.sh
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ █
```

2. Write a shell script to archive all the created files (in above question 1) in the given directory. Move the archive to a given specified backup directory (different from the given directory). Insert this script as a cronjob using the crontab command, which needs to be run every 5 minutes.

Code:

```
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ cat ques2.sh
#!/bin/bash

tar -czf /mnt/c/college/upes/sem4/linux/lab/scripts/exp5/logs/archieve$RANDOM.tar.gz /mnt/c/college/upes/sem4/linux/lab/scripts/exp5/*.txt
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ █

#Archiving files every 5 mins
*/5 * * * * /mnt/c/college/upes/sem4/linux/lab/scripts/exp5/ques2.sh
```

Output:

```
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ ls
12775.txt 5741.txt ques1.sh ques2.sh
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ ls -l logs
total 0
-rwxrwxrwx 1 ayroid ayroid 303 Feb 27 22:40 archieve2499.tar.gz
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ █
```

3. Write a shell script to automatically delete the archive files (in the backup archive) that are older than 30 minutes.

Code:

```
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ cat ques3.sh
#!/bin/bash

find /mnt/c/college/upes/sem4/linux/lab/scripts/exp5/logs -mmin +30 -delete
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$

#Deleting old log archive files which are 30 mins older
* * * * * /mnt/c/college/upes/sem4/linux/lab/scripts/exp5/ques3.sh
```

Output (After 30 mins):

```
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ ls
12775.txt 15390.txt 28418.txt 5741.txt logs ques1.sh ques2.sh ques3.sh
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ ls -l logs
total 0
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$
```

4. Monitor the disk usage of the given disk and alert if it is beyond the given threshold.

Code:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp5$ cat ques4.sh
#!/bin/bash

# Set the threshold for disk usage
THRESHOLD=90

# Get the disk usage as a percentage
USAGE=$(df -h | grep "/mnt/c" | awk '{print $5}' | cut -d '%' -f1)

# Check if disk usage is above the threshold
if [ $USAGE -gt $THRESHOLD ]; then
    # Send an alert
    echo "Disk usage is above $THRESHOLD% on /dev/sda1" | mail -s "Disk Alert" ayroids@gmail.com
fi

#Automating the disk usage threshold script to run every 30 mins
*/30 * * * * /mnt/c/college/upes/sem4/linux/lab/scripts/exp5/ques4.sh
```

5. Write a shell script to install the LAMP stack on Linux Machine (A-Apache, M-MySQL, P-PHP).

Code:

```
ayroid@Ayroid:/mnt/c/College/UPES/Sem4/Linux/Lab/scripts/exp5$ cat ques5.sh
#!/bin/bash

# Update the system
sudo apt update
sudo apt upgrade -y

# Install Apache
sudo apt install apache2 -y

# Install MySQL
sudo apt install mysql-server -y
sudo mysql_secure_installation

# Install PHP
sudo apt install php libapache2-mod-php php-mysql -y

# Restart Apache
sudo systemctl restart apache2
```

EXPERIMENT-6

Automation Scripts to prevent Errors

1. Write a shell script to search and replace a given text input by the user with a new text across multiple files.

Code:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ cat ques1.sh
#!/bin/bash

read -p "Enter the text to search for: " SEARCH_TEXT
read -p "Enter the text to replace it with: " REPLACE_TEXT

FILES=$(find . -type f)

for FILE in $FILES; do
    grep -q "$SEARCH_TEXT" $FILE

    if [ $? -eq 0 ]; then
        sed -i "s/$SEARCH_TEXT/$REPLACE_TEXT/g" $FILE
        echo "Replaced \"$SEARCH_TEXT\" with \"$REPLACE_TEXT\" in $FILE"
    fi
done
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ cat a.txt
The moon is one of the most fascinating objects in the sky. It is a natural satellite that orbits around the Earth and can be seen from almost every part of the world. The moon is approximately one-quarter the size of the Earth and has a rocky, barren surface covered in craters, mountains, and valleys.

The moon's gravitational pull affects ocean tides, and its phases have been used to track time and calendar dates for centuries. The moon has also been the subject of many myths and legends across different cultures.

In recent years, space exploration has brought about new discoveries about the moon, including evidence of water on its surface. The moon also remains a destination for future space missions, with plans to establish a permanent human presence on the moon in the coming decades.

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ ./ques1.sh
Enter the text to search for: moon
Enter the text to replace it with: star
Replaced "moon" with "star" in ./a.txt
Replaced "moon" with "star" in ./b.txt
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ cat a.txt
The star is one of the most fascinating objects in the sky. It is a natural satellite that orbits around the Earth and can be seen from almost every part of the world. The star is approximately one-quarter the size of the Earth and has a rocky, barren surface covered in craters, mountains, and valleys.

The star's gravitational pull affects ocean tides, and its phases have been used to track time and calendar dates for centuries. The star has also been the subject of many myths and legends across different cultures.

In recent years, space exploration has brought about new discoveries about the star, including evidence of water on its surface. The star also remains a destination for future space missions, with plans to establish a permanent human presence on the star in the coming decades.
```

2. Write a shell script to check whether the given input is a valid Alphanumeric character.

Code:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ cat ques2.sh
#!/bin/bash

read -p "Enter a character: " char

if [[ "$char" =~ ^[[:alnum:]]+$ ]]; then
    echo "The input is a valid alphanumeric character."
else
    echo "The input is not a valid alphanumeric character."
fi
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ ./ques2.sh
Enter a character: 2
The input is a valid alphanumeric character.
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ ./ques2.sh
Enter a character: *
The input is not a valid alphanumeric character.
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ ./ques2.sh
Enter a character: 7
The input is a valid alphanumeric character.
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ ./ques2.sh
Enter a character: -
The input is not a valid alphanumeric character.
```

3. Write a shell script to transform the given input text to lowercase, uppercase and first letter uppercase.

Code:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ cat ques3.sh
#!/bin/bash

read -p "Enter some text: " text

echo "Lowercase: ${text,,}"
echo "Uppercase: ${text^^}"
echo "First letter uppercase: ${text^}"
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$ ./ques3.sh
Enter some text: ayush Singh KUSHWAH
Lowercase: ayush singh kushwah
Uppercase: AYUSH SINGH KUSHWAH
First letter uppercase: Ayush Singh KUSHWAH
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp6$
```

EXPERIMENT-7

Working with Make and Makefile

1. Write a C program for calculator with basic functionality subtraction, addition, division, and multiplication. Each functionality (addition, subtraction, etc.) should be written in separate source files along with a single main source file. Use make utility to create an executable file for the calculator.

Functionality Codes:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7a$ cat add.c
#include<stdio.h>
#include"header.h"
void add(int a, int b){
    printf("%d",a+b);
}
```

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7a$ cat sub.c
#include<stdio.h>
#include"header.h"
void sub(int a, int b){
    printf("%d",a-b);
}
```

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7a$ cat mul.c
#include<stdio.h>
#include"header.h"
void mul(int a, int b){
    printf("%d",a*b);
}
```

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7a$ cat div.c
#include<stdio.h>
#include"header.h"
void div(int a, int b){
    printf("%d",a/b);
}
```

Header file:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7a$ cat header.h
void add(int a, int b);
void sub(int a, int b);
void mul(int a, int b);
void div(int a, int b);
```

Main file:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7a$ cat main.c
#include<stdio.h>
#include"header.h"

int main(){
    int a, b;
    char op;
    printf("Enter 2 Numbers: ");
    scanf("%d %d", &a, &b);
    printf("Enter Operation (+, -, *, /): ");
    scanf(" %c",&op);

    switch (op){
        case '+':
            add(a,b);
            break;
        case '-':
            sub(a,b);
            break;
        case '*':
            mul(a,b);
            break;
        case '/':
            div(a,b);
            break;
        default:
            printf("Invalid Operation.\n");
            return 1;
    }
    printf("\n");
    return 0;
}
```

Make file:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7a$ cat makefile
calculator: main.o add.o sub.o mul.o div.o
    gcc main.o add.o sub.o mul.o div.o -o calculator

clean:
    rm -rf *.o final
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7a$ make calculator
cc -c -o main.o main.c
cc -c -o add.o add.c
cc -c -o sub.o sub.c
cc -c -o mul.o mul.c
cc -c -o div.o div.c
gcc main.o add.o sub.o mul.o div.o -o calculator
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7a$ ./calculator
Enter 2 Numbers: 2 3
Enter Operation (+, -, *, /): *
6
```

2. In the above program include two additional features for computing power and modulus of given numbers in the calculator program. Use Make utility to automatically build the second version of the calculator with additional features included in it.

New Functionality Codes:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7b$ cat mod.c
#include<stdio.h>
#include"header.h"
void mod(int a, int b){
    printf("%d",a%b);
}
```

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7b$ cat powe.c
#include<stdio.h>
#include<math.h>
#include"header.h"

void powe(int a, int b){
    int x=a;
    for(int i=1;i<b;i++){
        x=x*a;
    }
    printf("%d", x);
}
```

Updated Header file:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7b$ cat header.h
void add(int a, int b);
void sub(int a, int b);
void mul(int a, int b);
void div(int a, int b);
void mod(int a, int b);
void powe(int a, int b);
```

Updated Main file:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7b$ cat main.c
#include<stdio.h>
#include"header.h"

int main(){
    int a, b;
    char op;
    printf("Enter 2 Numbers: ");
    scanf("%d %d", &a, &b);
    printf("Enter Operation (+,-,*,/,^,%): ");
    scanf(" %c",&op);

    switch (op){
        case '+':
            add(a,b);
            break;
        case '-':
            sub(a,b);
            break;
        case '*':
            mul(a,b);
            break;
        case '/':
            div(a,b);
            break;
        case '^':
            powe(a,b);
            break;
        case '%':
            mod(a,b);
            break;
        default:
            printf("Invalid Operation.\n");
            return 1;
    }
    printf("\n");
    return 0;
}
```

Updated Make file:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7b$ cat makefile
calculator: main.o add.o sub.o mul.o div.o
    gcc main.o add.o sub.o mul.o div.o -o calculator

calculator2: main.o add.o sub.o mul.o div.o mod.o powe.o
    gcc main.o add.o sub.o mul.o div.o mod.o powe.o -o calculator2

clean:
    rm -rf *.o final
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7b$ make calculator2
cc -c -o main.o main.c
cc -c -o add.o add.c
cc -c -o sub.o sub.c
cc -c -o mul.o mul.c
cc -c -o div.o div.c
cc -c -o mod.o mod.c
cc -c -o powe.o powe.c
gcc main.o add.o sub.o mul.o div.o mod.o powe.o -o calculator2
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7b$ ./calculator2
Enter 2 Numbers: 3 4
Enter Operation (+,-,*,/,^,%): ^
81
```

3. Write Makefile in above program using the best practices.

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7b$ cat makefile
.PHONY = clean
CC = gcc
OBJECTS1 = main.o add.o sub.o mul.o div.o
OBJECTS2 = main.o add.o sub.o mul.o div.o mod.o powe.o

calculator: $(OBJECTS1)
    $(CC) -o calculator $(OBJECTS1)

calculator2: $(OBJECTS2)
    $(CC) -o calculator2 $(OBJECTS2)

clean:
    rm -rf *.o final
```

4. Use make utility to archive the logs file and move it to some other location.

Code:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7/ques4$ cat makefile
LOGS = /var/log/*.log
ARCHIVE = /mnt/c/college/upes/sem4/linux/lab/scripts/exp7/ques4/
FILENAME = archive-$(shell date +%Y%m%d).tar.gz

execute: archive delete

archive:
    tar -czvf $(ARCHIVE)$(FILENAME) $(LOGS)
    echo "Archive Successful!"

delete:
    rm -f $(LOGS)
    echo "Logs Deleted!"
```


Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7/ques4$ make execute
tar -czvf /mnt/c/college/upes/sem4/linux/lab/scripts/exp7/ques4/archive-20230322.tar.gz /var/log/*.log
tar: Removing leading '/' from member names
/var/log/alternatives.log
tar: Removing leading '/' from hard link targets
/var/log/bootstrap.log
/var/log/dpkg.log
/var/log/fontconfig.log
/var/log/ubuntu-advantage.log
/var/log/upgrade-policy-changed.log
echo "Archive Successful!"
Archive Successful!
rm -f /var/log/*.log
rm: cannot remove '/var/log/alternatives.log': Permission denied
rm: cannot remove '/var/log/bootstrap.log': Permission denied
rm: cannot remove '/var/log/dpkg.log': Permission denied
rm: cannot remove '/var/log/fontconfig.log': Permission denied
rm: cannot remove '/var/log/ubuntu-advantage.log': Permission denied
rm: cannot remove '/var/log/upgrade-policy-changed.log': Permission denied
make: *** [makefile:12: delete] Error 1
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7/ques4$ ls
archive-20230322.tar.gz makefile
```

5. Demonstrate the usage of conditional statements in a Makefile for a simple C/shell script program.

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7/ques5$ cat makefile
all: hello

ifdef DEBUG
CFLAGS=-g
endif

hello: hello.c
    gcc $(CFLAGS) -o hello hello.c

clean:
    rm -f hello
```

Output:

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7/ques5$ make
gcc -o hello hello.c
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp7/ques5$ ./hello
Hello, world!
```

EXPERIMENT-8

Working with Make and Makefile

1. Write a command to find out the log files and store it in another file using a single command.

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques1$ ls
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques1$ find /var/log -name '*.log' -type f -exec cp {}
/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques1/ \;
find: '/var/log/private': Permission denied
find: '/var/log/unattended-upgrades': Permission denied
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques1$ ls
alternatives.log  dpkg.log          history.log        ubuntu-advantage.log
bootstrap.log     fontconfig.log    term.log          upgrade-policy-changed.log
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques1$
```

2. Write a program in C where there are two parts. The first part prints “Hello World” message. Store it in a file. The second part of the program directly display the error message of division by zero. Store the error message in another file.

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques2$ cat ques2.c
#include <stdio.h>
int main() {

    FILE *helloFile = fopen("hello.txt", "w");
    fprintf(helloFile, "Hello World\n");
    fclose(helloFile);

    int num = 1;
    int divisor = 0;
    FILE *errorFile = fopen("error.txt", "w");
    if (divisor == 0) {
        fprintf(errorFile, "Error: Division by zero\n");
    }
    fclose(errorFile);

    return 0;
}
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques2$ gcc ques2.c -o ques2
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques2$ ./ques2
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques2$ ls
error.txt  hello.txt  ques2  ques2.c
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques2$ cat error.txt
Error: Division by zero
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques2$ cat hello.txt
Hello World
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques2$
```

3. Write a program in C where there are two parts. The first part prints “Hello World” message. The second part of the program display the error message of division by zero. Store the two messages into a single file.

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques3$ cat ques3.c
#include <stdio.h>
int main() {

    FILE *File = fopen("hello.txt", "a");
    fprintf(File, "Hello World\n");

    int num = 1;
    int divisor = 0;
    if (divisor == 0) {
        fprintf(File, "Error: Division by zero\n");
    }
    fclose(File);

    return 0;
}
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques3$ gcc ques3.c -o ques3
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques3$ ./ques3
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques3$ ls
hello.txt  ques3  ques3.c
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques3$ cat hello.txt
Hello World
Error: Division by zero
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques3$
```

4. Write a program in C where we can take input of two numbers and by using switch case we can do addition, subtraction, multiplication, and division. If the switch case is not '+', '-', '*' or '/' the some error is occurred. Store the error into a file. The whole switch case is done within while(1) loop. So, in future if some another error is occurred then add the errors into previously created error file.

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques4$ cat ques4.c
#include <stdio.h>

int main() {
    double num1, num2, result;
    char op;

    while (1) {
        printf("Enter two numbers and an operator (+, -, *, /): ");
        scanf("%lf %lf %c", &num1, &num2, &op);

        switch (op) {
            case '+':
                result = num1 + num2;
                printf("%.2lf\n", result);
                break;
            case '-':
                result = num1 - num2;
                printf("%.2lf\n", result);
                break;
            case '*':
                result = num1 * num2;
                printf("%.2lf\n", result);
                break;
            case '/':
                if (num2 == 0) {
                    printf("Error: Division by zero\n");
                    FILE *errorFile = fopen("error.txt", "a");
                    fprintf(errorFile, "Error: Division by zero\n");
                    fclose(errorFile);
                } else {
                    result = num1 / num2;
                    printf("%.2lf\n", result);
                }
                break;
        }
    }
}
```

```

    default:
        printf("Error: Invalid operator\n");
        FILE *errorFile = fopen("error.txt", "a");
        fprintf(errorFile, "Error: Invalid operator\n");
        fclose(errorFile);
        break;
    }
}

return 0;
}

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques4$ gcc ques4.c -o ques4
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques4$ ./ques4
Enter two numbers and an operator (+, -, *, /): 1 4 *
4.00
Enter two numbers and an operator (+, -, *, /): 1 0 /
Error: Division by zero
Enter two numbers and an operator (+, -, *, /): 2 4 ^
Error: Invalid operator
Enter two numbers and an operator (+, -, *, /): 8 2 /
4.00
Enter two numbers and an operator (+, -, *, /): ^C
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques4$ cat error.txt
Error: Division by zero
Error: Invalid operator
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques4$

```

5. Try different Linux exit codes from the lab material.

- **exit 0:** exiting without any error.

```

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ cat exit1.sh
#!/bin/bash

echo "Hello, world!"
exit 0

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ ./exit1.sh
Hello, world!
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ echo $?
0
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$

```

- **exit 1:** catching all general errors.

```

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ cat exit1.sh
#!/bin/bash

echo "An error occurred."
exit 1

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ ./exit1.sh
An error occurred.
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ echo $?
1
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$

```

- **exit 126:** command invoked cannot execute.

```

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ cat exit126.sh
#!/bin/bash

# Try to execute a non-executable file
./non_executable_file.sh
exit 126

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ ./exit126.sh
./exit126.sh: line 4: ./non_executable_file.sh: No such file or directory
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$

```

- exit 127: command not found.

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ cat exit127.sh
#!/bin/bash

# Try to execute a command that does not exist
unknown_command
exit 127

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ ./exit127.sh
./exit127.sh: line 4: unknown_command: command not found
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$
```

- exit 128: invalid argument to exit.

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ cat exit128.sh
#!/bin/bash

# Attempt to run the script with incorrect arguments
if [ $# -lt 2 ]; then
    echo "Usage: $0 arg1 arg2"
    exit 128
fi

echo "Arguments: $1 and $2"

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ ./exit128.sh
Usage: ./exit128.sh arg1 arg2
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$
```

- exit 130: script terminated by ctrl-c.

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ cat exit130.sh
#!/bin/bash

# Trap the SIGINT signal (CTRL-C) and exit with exit code 130
trap 'echo "Interrupted by user"; exit 130' INT

# Wait for user input
read -p "Enter something: " input

# Print the user input
echo "You entered: $input"

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ ./exit130.sh
Enter something: 2
You entered: 2
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ ./exit130.sh
Enter something: ^CInterrupted by user
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$
```

- exit 255: exit status out of range.

```
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ cat exit255.sh
#!/bin/bash

# Check whether a required command is available
if ! command -v some_command &> /dev/null; then
    echo "Error: Required command 'some_command' not found"
    exit 255
fi

# Run the command
some_command --option argument

ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$ ./exit255.sh
Error: Required command 'some_command' not found
ayroid@Ayroid:/mnt/c/college/upes/sem4/linux/lab/scripts/exp8/ques5$
```

EXPERIMENT-9

1. Write two script files. In the first program some print message is written. Call the print message from the second script program.

```
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques1$ cat first.sh
#!/bin/bash
message() {
    echo "It's the First script!"
}
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques1$ cat second.s
h
#!/bin/bash
echo "It's the Second script!"
source ./first.sh
message
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques1$ ./second.sh
It's the Second script!
It's the First script!
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques1$
```

2. Write two script files. In the first program two functions are written. The two functions are uppercase and lowercase. Call the two functions from the second script program.

```
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques2$ cat first.sh
#!/bin/bash

uppercase(){
    echo $(echo $1 | tr '[:lower:]' '[:upper:]')
}

lowercase(){
    echo $(echo $1 | tr '[:upper:]' '[:lower:]')
}
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques2$ cat second.s
h
#!/bin/bash

source ./first.sh
echo "UPPERCASE CALLED!"
uppercase ayroid
echo "LOWERCASE CALLED!"
lowercase AYROID
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques2$ ./second.sh
UPPERCASE CALLED!
AYROID
LOWERCASE CALLED!
ayroid
```

3. Write two script files. In the first program four functions are written. The functions are converting a text to different color (red, yellow, green and blue). Call the four functions from the second script program.

```
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques3$ cat first.sh

#!/bin/bash
red_func(){
    echo -e "\033[31m$1\033[0m"
}
green_func(){
    echo -e "\033[32m$1\033[0m"
}
yellow_func(){
    echo -e "\033[33m$1\033[0m"
}
blue_func(){
    echo -e "\033[34m$1\033[0m"
}
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques3$ cat second.s
h
#!/bin/bash

source ./first.sh
red_func blue
blue_func yellow
green_func red
yellow_func green
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques3$ ./second.sh
blue
yellow
red
green
```

4. Try different script codes from the lab material.

```
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques4$ cat first.sh

#!/bin/bash

red_func(){
    echo -e "\033[31m$1\033[0m"
}
green_func(){
    echo -e "\033[32m$1\033[0m"
}
yellow_func(){
    echo -e "\033[33m$1\033[0m"
}
blue_func(){
    echo -e "\033[34m$1\033[0m"
}

bars(){
if [ -z "$1" ];
then
echo "Enter duration!!" >&2;
exit 1;
fi
```



```
d=$1
inc=$((100/$d))
for (( t=0; t<=100; t=t+inc ));
do
    for (( done=0; done<t; done=done+1 ));
    do
        printf "${green_func}*";
        done
        for ((tr=t; tr<100; tr=tr+1));
        do
            printf "${red_func}-";
            done
            printf "| ${yellow_func}$t) %%" ;
            sleep 1
            printf "\r";
        done
    echo -e "\n";
}
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques4$ cat second.sh
#!/bin/bash

source ./first.sh
red_func Ayroid
yellow_func Ayroid
green_func Ayroid
bars 10
ayush@ayroid-Victus:/mnt/Storage/College/SEM4/Linux-Lab/exp9/ques4$ ./second.sh
Ayroid
Ayroid
Ayroid
-----
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```